Automatic Change-Point Detection in Time Series via Deep Learning

A PyTorch Reimplementation and Extension of Li et al. [5]

P.C.L.N. Vũ N.T.T. Hoài N.N. Quang

VNU-HCM University of Science
Faculty of Mathematics and Informatics

Advisor: Tô Đức Khánh

April 2026

Outline

Introduction

Theoretical Foundation

Methodology

Experiments & Results

Conclusion

Change-Point Detection

Definition

Given a time series $X_1, X_2, \ldots, X_n$, identify points τ where the statistical properties (mean, variance, slope) change abruptly.

Two fundamental questions:

- Does a change exist at all?
- If so, *where* does it occur?

Real-world applications [7]:

- Signal processing & climate forecasting
- Cell biology (gene expression shifts)
- Human motion tracking (activity transitions)
- Financial market regime changes

Classical Approach: CUSUM and Its Limits

Hypothesis test:

$$H_0 : \mu_1 = \mu_2 = \cdots = \mu_n \qquad \text{vs.} \qquad H_1 : \exists\, \tau \text{ s.t. } \mu_{\leq \tau} \neq \mu_{>\tau}$$

CUSUM statistic [6]:

$$T_n = \max_{1 \leq k < n} \sqrt{\frac{k(n-k)}{n}} \left| \bar{X}_{1:k} - \bar{X}_{k+1:n} \right|$$

Reject H_0 when $T_n > \lambda$; estimated change-point $\hat{\tau} = \arg\max$.

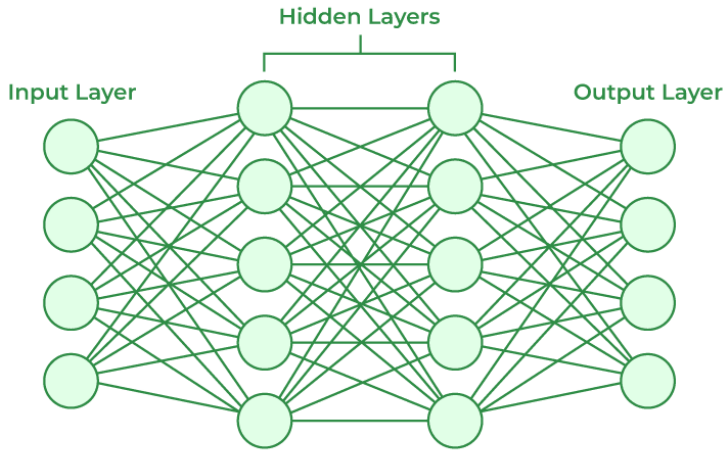
Key limitations [5]:

- Requires **i.i.d. Gaussian** noise — collapses to random guessing under auto-correlated or heavy-tailed distributions
- Manually designing optimal statistics for complex signals is impractical

Deep Neural Network Architectures

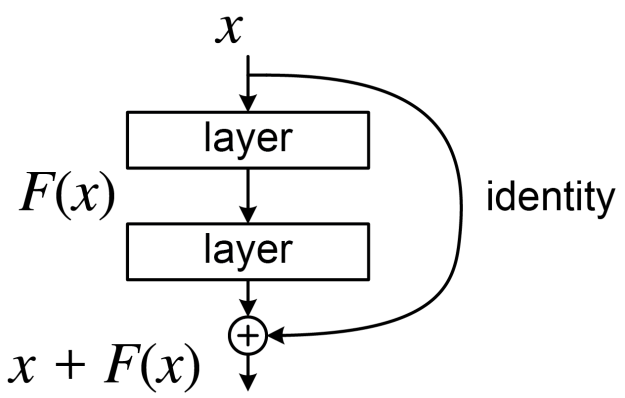
Multi-Layer Perceptron (MLP) [5]

- L hidden layers, width m_ℓ
- Input: sequence $X \in \mathbb{R}^n$
- Output: $\hat{Y} \in \{0, 1\}$ (change / no change)
- Activation: shifted ReLU σ_b



Residual CNN (ResCNN) [1]

- Skip connections prevent vanishing gradients
- Each block: $y = \mathcal{F}(x, \{W_i\}) + x$
- 21 residual blocks, base channels $= 32$
- Handles complex noise automatically



Notation for Theoretical Results

Symbol	Meaning
n	Sequence length
B	Minimum SNR parameter (defines class $\Theta(B)$); actual signal $\lvert\mu_L - \mu_R\rvert\sqrt{\eta(1-\eta)} \geq B,\ \ \eta = \tau/n$
N	Training set size
$\delta \in (0,1)$	Confidence parameter (failure probability)
C	Universal constant
L	Number of hidden layers
h_{ERM}	Empirical risk minimiser (trained DNN)
$\mathcal{R}(\cdot)$	Misclassification risk $= P(h \neq Y)$

Key Theoretical Results [5]

Theorem 1 — MLP $\equiv$ CUSUM

A single-hidden-layer MLP with $2n-2$ **ReLU units** can *exactly represent* the CUSUM transformation.

$$\mathcal{R}(h_{\text{ERM}}) \;\leq\; ne^{-nB^2/8} + C\sqrt{\frac{n^2 \log n \cdot \log N + \log(1/\delta)}{N}}$$

Error $\to 0$ when $N \gg n^2 \log n$.

Theorem 2 — Pruned Deep MLP

An L-layer MLP with only $4\lfloor \log_2 \mathrm{n} \rfloor$ units per layer achieves:

$$\mathcal{R}(h_{\text{ERM}}) \;\leq\; 2\lfloor \log_2 n \rfloor\, e^{-nB^2/24} + C\sqrt{\frac{L^2 n \log^2(Ln) \log N + \log(1/\delta)}{N}}$$

Depth trades width for better generalization with fewer training samples.

Problem Reformulation [5]

Insight: Convert hypothesis testing $\rightarrow$ binary classification.

$$\psi : X \mapsto Y \in \{0, 1\} \quad \begin{cases} 0 & \text{no change point} \\ 1 & \text{change point present} \end{cases}$$

Four synthetic noise scenarios (sequence length $n = 100$):

Scenario	Noise type	Challenge
S1	i.i.d. Gaussian $\mathcal{N}(0, 1)$	Baseline (CUSUM-friendly)
S1'	AR(1), $\rho = 0.7$	Fixed auto-correlation
S2	AR(1), $\rho_t \sim$ Unif$[0, 1]$	Time-varying dependence
S3	Cauchy (heavy-tailed)	Infinite variance

Data augmentation: reverse each sequence to double training size.

Normalization: min-max (S1/S1'/S2); trimmed robust (S3).

Training Configuration

Input: raw sequence $x_t \in \mathbb{R}^n$ (canonical runs use no pre-transformations). Optional extensions — squared x_t^2 and cross-products $x_t \cdot x_{t+1}$ — are available in code but disabled in all reported experiments.

Loss: Binary Cross-Entropy with logits

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

Hyperparameter	Value
Optimizer	Adam [3] ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Learning rate	0.001
Epochs / Batch size	200 / 32
Early stopping patience	20 epochs
MLP width	$4\lfloor \log_2 n \rfloor$ units/layer
ResCNN dropout	0.3

Multi-Change-Point Localization (Algorithm 1) [5]

Extends binary classifier to localize *multiple* change points in long series.

1. **Sliding window:** apply classifier to every window of size n along the full series
2. **MOSUM-style smoothing:** compute rolling average of predicted probabilities $\bar{p}_t$
3. **Segment detection:** identify maximal contiguous segments where $\bar{p}_t \geq \gamma$ (threshold $\gamma = 0.5$)
4. **Localization:** within each segment, report $\hat{\tau} = \arg\max_t \bar{p}_t$ as the estimated change point

Note on HASC

The main HASC result (Experiment 3) is **30-class activity classification**, not localization. Algorithm 1 is an optional extension for binary change-point localization in longer sequences.

Experiment 1: Single Change-Point on Synthetic Data

Setup [5, 4]: $n = 100$, $N_{\text{train}} = 10{,}000$ / $N_{\text{test}} = 2{,}000$ (50/50 class balance), fixed splits with SHA-256 integrity hashes for reproducibility.

	Detection Accuracy			
Scenario	MLP	ResCNN	CUSUM	Winner
S1 (Gaussian)	0.9190	**0.9255**	0.8915	ResCNN
S1' (AR $\rho = 0.7$)	0.7290	**0.7435**	0.5005	ResCNN
S2 (AR time-vary)	**0.7540**	0.7525	0.5000	MLP
S3 (Cauchy)	0.5970	**0.9475**	0.5110	ResCNN

Key takeaways:

- S1: all methods converge — confirms theory
- S1'/S2: CUSUM $\approx$ random guessing; DNNs maintain $> 72\%$
- S3: ResCNN **+44 pp** over CUSUM — strongest result

Experiment 3: Real-World HASC Dataset [2]

Task: Classify 7-second accelerometer windows into 30 activity states (6 pure + 24 transition activities).

Setup

- Window size: $n = 700$ samples
- 3-axis input (ResCNN multi-channel)
- Training: 13 380 windows, persons 101–105*
- 30-class softmax output

Results (best at epoch 121)

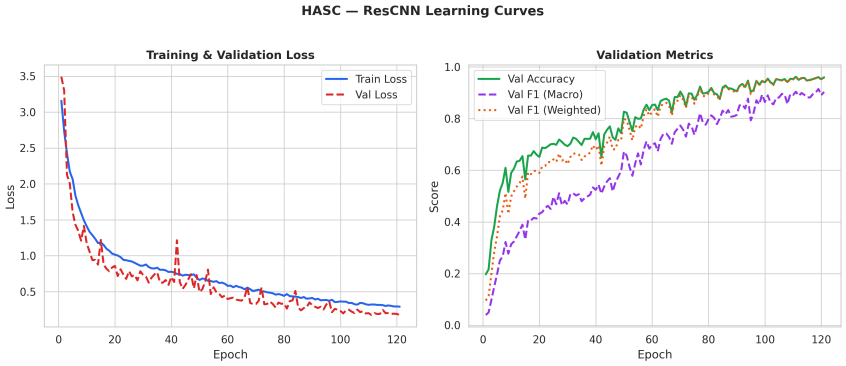
- Val. loss: 0.1749
- Val. accuracy: 96.21%
- Macro F1: 0.9042
- Weighted F1: 0.9608

*Reported checkpoint provenance; current split metadata may reflect later reruns.

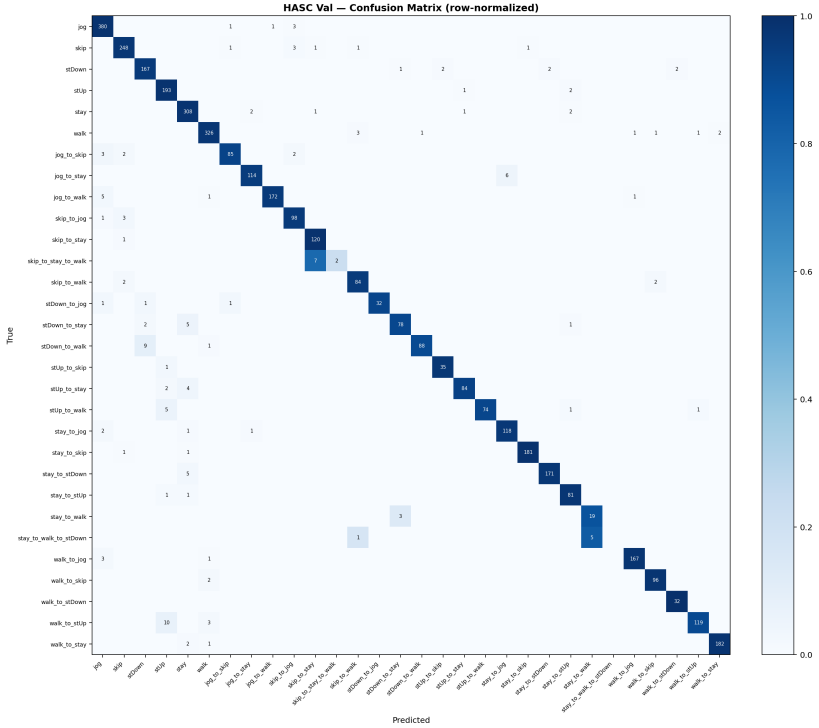
Key finding

The same ResCNN architecture used for binary synthetic classification scales directly to 30-class real-world sensor data with no architectural redesign.

HASC Training Dynamics & Confusion Matrix



Fast convergence; stable val. loss after $\sim$50 epochs; F1 $>$ 0.90 consistently.



30$\times$30 confusion matrix (4006 val. samples).
Most classes: near-perfect recall.
Rare transitions ($<$10 samples) show lower recall.

Conclusion — Contributions

- **Reproducible PyTorch reimplementation** of [5] with deterministic synthetic datasets and SHA-256 integrity checks
- **Validated theory–practice alignment:** MLP/ResCNN match CUSUM on Gaussian data and substantially outperform on complex noise (up to +44 pp on Cauchy distributions)
- **Real-world scaling:** same ResCNN achieves **96.21%** accuracy on 30-class HASC [2] activity recognition

Deep learning is not just a substitute for classical statistics — it adapts where classical methods break down.

Limitations & Future Work

- **Multi-CP localization** (Algorithm 1) demonstrated qualitatively only — quantified $|\hat{\tau} - \tau|$ error over many sequences is needed
- **Limited baselines:** only CUSUM [6] compared; non-parametric and robust methods (e.g. PELT, E-divisive) should be added
- **No ablation studies** on pre-transformations, robust normalization, or ResCNN depth; HASC evaluation needs person-held-out splits

References I

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun.
Deep residual learning for image recognition.
In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

Nobuo Kawaguchi, Nobuhiro Ogawa, Yohei Iwasaki, Katsuhiko Kaji, Tsutomu Terada, Kazuya Murao, Sozo Inoue, Yoshihiro Kawahara, Yasuyuki Sumi, and Nobuhiko Nishio.
HASC challenge: Gathering large scale human activity corpus for the real-world activity understandings.
In *Proceedings of the 2nd Augmented Human International Conference (AH '11)*, 2011.

Diederik P. Kingma and Jimmy Ba.
Adam: A method for stochastic optimization.
In *3rd International Conference on Learning Representations (ICLR)*, 2015.

References II

Jie Li.
AutoCPD: Reference implementation.
https://github.com/Jieli12/AutoCPD, 2024.
Accessed: 2026-04.

Jie Li, Paul Fearnhead, Piotr Fryzlewicz, and Tengyao Wang.
Automatic change-point detection in time series via deep learning.
Journal of the Royal Statistical Society Series B: Statistical Methodology, 86(2):273–285, 2024.

E. S. Page.
Continuous inspection schemes.
Biometrika, 41(1/2):100–115, 1954.

Charles Truong, Laurent Oudre, and Nicolas Vayatis.
Selective review of offline change point detection methods.
Signal Processing, 167:107299, 2020.

Thank You

Questions?

Code: `github.com/ngnquanq/change_point_detection`